File: features.py

```
import numpy as np

# Landmark mapping based on MediaPipe Hands
FINGER_JOINTS = {
    "thumb": {"mcp": (0, 1, 2), "pip": (1, 2, 3), "dip": (2, 3, 4)},
    "index": {"mcp": (0, 5, 6), "pip": (5, 6, 7), "dip": (6, 7, 8)},
    "middle": {"mcp": (0, 9, 10), "pip": (9, 10, 11), "dip": (10, 11, 12)},
    "ring": {"mcp": (0, 13, 14), "pip": (13, 14, 15), "dip": (14, 15, 16)},
    "pinky": {"mcp": (0, 17, 18), "pip": (17, 18, 19), "dip": (18, 19, 20)}
}

def angle_between_points(a, b, c):
    """
    Calculates the angle at point 'b' given points 'a', 'b', 'c'.
    Points are expected to be (x, y, z) tuples or numpy arrays.
    """
    ba = np.array(a) - np.array(b)
    bc = np.array(c) - np.array(b)

    norm_ba = np.linalg.norm(ba)
    norm_bc = np.linalg.norm(bc)

    if norm_ba < 1e-8 or norm_bc < 1e-8:
        return 180.0  # Default to straight if points are co-located

    cosine = np.dot(ba, bc) / (norm_ba * norm_bc)
    cosine = np.clip(cosine, -1.0, 1.0)

    return np.degrees(np.arccos(cosine))

def extract_finger_features(landmarks):
    """
    Given a list of 21 landmarks (x, y, z), calculate joint angles and raw curl scores.
    """
    features = {}

    for finger_name, joints in FINGER_JOINTS.items():
        # Calculate angles
        mcp_angle = angle_between_points(landmarks[joints["mcp"][0]],
                                         landmarks[joints["mcp"][1]],
                                         landmarks[joints["mcp"][2]])

        pip_angle = angle_between_points(landmarks[joints["pip"][0]],
                                         landmarks[joints["pip"][1]],
                                         landmarks[joints["pip"][2]])

        dip_angle = angle_between_points(landmarks[joints["dip"][0]],
                                         landmarks[joints["dip"][1]],
                                         landmarks[joints["dip"][2]])

        # Bend is defined as how far it is from straight (180 degrees)
        mcp_bend = 180.0 - mcp_angle
        pip_bend = 180.0 - pip_angle
        dip_bend = 180.0 - dip_angle

        # Weighted raw curl
        raw_curl = 0.25 * mcp_bend + 0.50 * pip_bend + 0.25 * dip_bend

        features[finger_name] = {
            "angles": {
                "mcp": float(mcp_angle),
                "pip": float(pip_angle),
                "dip": float(dip_angle)
            },
            "raw_curl": float(raw_curl)
```

```
        }

    return features

def normalize_landmarks(landmarks):
    """
    Normalizes landmarks relative to the wrist (landmark 0).
    """
    if not landmarks:
        return []

    wrist = np.array(landmarks[0])
    normalized = []

    for lm in landmarks:
        norm_point = np.array(lm) - wrist
        normalized.append((float(norm_point[0]), float(norm_point[1]), float(norm_point[2])))

    return normalized
```

File: filters.py

```
import time

class ExponentialMovingAverage:
    def __init__(self, alpha=0.3):
        self.alpha = alpha
        self.state = None

    def update(self, value):
        if self.state is None:
            self.state = value
        else:
            self.state = self.alpha * value + (1.0 - self.alpha) * self.state
        return self.state

    def reset(self):
        self.state = None

class HandStateFilter:
    def __init__(self, alpha_curl=0.3, timeout_seconds=0.3):
        self.alpha_curl = alpha_curl
        self.timeout_seconds = timeout_seconds

        self.last_valid_time = 0
        self.last_valid_state = None

        # Dict of EMA filters for each finger's curl
        self.curl_filters = {
            "thumb": ExponentialMovingAverage(alpha=alpha_curl),
            "index": ExponentialMovingAverage(alpha=alpha_curl),
            "middle": ExponentialMovingAverage(alpha=alpha_curl),
            "ring": ExponentialMovingAverage(alpha=alpha_curl),
            "pinky": ExponentialMovingAverage(alpha=alpha_curl)
        }

    def process(self, hand_detected, confidence, raw_features, current_time):
        """
        Applies filtering to the raw features and handles tracking loss/confidence drops.
        """
        # Define a safe neutral state
        neutral_state = {
            "thumb": {"curl": 0.0},
            "index": {"curl": 0.0},
            "middle": {"curl": 0.0},
            "ring": {"curl": 0.0},
            "pinky": {"curl": 0.0}
        }

        # If no hand or low confidence
        if not hand_detected or confidence < 0.5:
            time_since_last_valid = current_time - self.last_valid_time

            if time_since_last_valid < self.timeout_seconds and self.last_valid_state is not None:
                # Keep last valid state for a brief moment
                return self.last_valid_state
            else:
                # Fallback to safe neutral state, reset filters
                for f in self.curl_filters.values():
                    f.reset()
                self.last_valid_state = neutral_state
                return neutral_state

        # We have a valid hand, update filters
        filtered_state = {}
        for finger, features in raw_features.items():
            smoothed_curl = self.curl_filters[finger].update(features["raw_curl"])
```

```
        filtered_state[finger] = {
            "curl": smoothed_curl,
            # we can also pass through raw angles if needed
            "angle": features["angles"]["pip"] # using PIP as a representative angle for the fin
al output format
        }

    self.last_valid_time = current_time
    self.last_valid_state = filtered_state
    return filtered_state
```

File: calibration.py

```
import numpy as np

class CalibrationManager:
    def __init__(self, default_open=0.0, default_closed=90.0):
        # We store the expected raw curl values for an open hand vs closed fist.
        # These default values are rough estimates (bend degrees).
        self.baselines = {
            "thumb": {"open": default_open, "closed": default_closed},
            "index": {"open": default_open, "closed": default_closed},
            "middle": {"open": default_open, "closed": default_closed},
            "ring": {"open": default_open, "closed": default_closed},
            "pinky": {"open": default_open, "closed": default_closed}
        }

    def calibrate(self, finger, pose, raw_curl_value):
        """
        pose should be 'open' or 'closed'
        """
        if pose in ["open", "closed"]:
            self.baselines[finger][pose] = raw_curl_value

    def normalize_curl(self, finger, raw_curl):
        """
        Maps the raw curl value to a 0.0 - 1.0 range based on calibrated baselines.
        """
        open_baseline = self.baselines[finger]["open"]
        closed_baseline = self.baselines[finger]["closed"]

        # Avoid division by zero
        if abs(closed_baseline - open_baseline) < 1e-5:
            return 0.0

        normalized = (raw_curl - open_baseline) / (closed_baseline - open_baseline)
        return float(np.clip(normalized, 0.0, 1.0))
```

File: hand_tracker.py

```
import cv2
import mediapipe as mp

class HandTracker:
    def __init__(self, max_num_hands=1, model_complexity=1,
                 min_detection_confidence=0.6, min_tracking_confidence=0.6):
        self.mp_hands = mp.solutions.hands
        self.hands = self.mp_hands.Hands(
            static_image_mode=False,
            max_num_hands=max_num_hands,
            model_complexity=model_complexity,
            min_detection_confidence=min_detection_confidence,
            min_tracking_confidence=min_tracking_confidence
        )

    def process_frame(self, frame):
        # MediaPipe expects RGB images
        rgb_frame = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
        results = self.hands.process(rgb_frame)

        frame_height, frame_width, _ = frame.shape

        output = {
            "hand_detected": False,
            "confidence": 0.0,
            "handedness": None,
            "landmarks_norm": [],
            "landmarks_pixel": []
        }

        if results.multi_hand_landmarks and results.multi_handedness:
            # For Phase 1, we only process the first hand detected
            hand_landmarks = results.multi_hand_landmarks[0]
            handedness_info = results.multi_handedness[0].classification[0]

            output["hand_detected"] = True
            output["confidence"] = handedness_info.score
            # MediaPipe's handedness is often flipped with front-facing webcams
            label = handedness_info.label
            output["handedness"] = "Right" if label == "Left" else "Left"

            for lm in hand_landmarks.landmark:
                # Normalized coordinates (0 to 1)
                output["landmarks_norm"].append((lm.x, lm.y, lm.z))
                # Pixel coordinates
                output["landmarks_pixel"].append((
                    int(lm.x * frame_width),
                    int(lm.y * frame_height)
                ))

        return output

    def close(self):
        self.hands.close()
```